Contents

STAT 5243 Project 3 — A/B Test on Data-Cleaning UX

GitHub Repo: https://github.com/ZemingLiang/STAT5243-Project3-Team21

Group Members: Bohong Zheng (bz2575), Zeming Liang (zl3688), Zuer Weng (zw3118), Maya Rubin (mr4459)

Deployed App (single URL, in-app randomization): https://019d23ea-1266-cada-1d21-45e5d97e6ea5.share.connect.posit.cloud/

---

1. Introduction and Research Question

In Project 2, our team built an interactive Shiny-for-Python data workbench for loading, cleaning, feature-engineering, and exploring tabular datasets. Usability testing revealed that the **Cleaning** tab is the step most new users find confusing: they frequently clicked **Apply** without first clicking **Preview**, overwriting the active dataset with an unintended transformation and then abandoning the session. Cleaning is also the tab with the most configurable options (nine operations, multiple strategies each), so any friction there disproportionately harms the end-to-end experience.

For Project 3 we keep the rest of the app unchanged and test a single interaction-design hypothesis on the Cleaning tab:

> **Research Question.** Does a guided, four-step workflow layout with a prominent "Preview, then Save" call-to-action box improve users' preview-before-apply behaviour on the Cleaning tab, compared to the original flat Preview/Apply button layout?

Primary hypotheses.

- **H0:** The guided layout (Version B) does not change the proportion of cleaning applies that are preceded by a preview within the same session, compared to the original layout (Version A).
- **H1:** The guided layout changes (in either direction) that proportion.

We also pre-registered three **secondary metrics** — apply-success rate, preview-to-apply conversion rate, and total successful cleaning actions per session — each with the same two-sided H0/H1 formulation. Multiple-comparison correction is applied across the family of four tests (see §4).

The test matters because data-cleaning UX friction is a common failure mode of code-free analytics tools, and quantifying the effect of a small, cheap copy / layout change gives the team a concrete basis for deciding whether to roll the guided layout forward to all new users.

---

2. Experimental Design and Methodology

2.1 Platform architecture

Rather than deploying two separate apps, we implemented both experimental arms inside a **single Shiny application** (`app_trt.py`). On session start the server assigns the session to one of two groups uniformly at random and then conditionally renders the Cleaning-tab UI based on that assignment. The rest of the app — Guide, Load, Overview, Feature Engineering, and EDA tabs — is identical across arms. Benefits:

- Both arms share one deployment URL, so the link we share with classmates and the public does not leak the group assignment.
- Routing is guaranteed 50/50 at the application layer rather than relying on an external load balancer.
- Event logging uses a single CSV schema, so downstream analysis is simpler.

Key implementation anchors in app_trt.py:

- Random assignment: `ab_group_state = reactive.value(random.choice(["A", "B"]))` (line 1365).
- Per-session identifier: `session_id = str(uuid.uuid4())` (line 1366).
- Conditional UI rendering: `cleaning_sidebar_intro()` and `cleaning_action_buttons()` (lines 1431–1498) select A vs. B markup based on `ab_group_state`.
- Event log writer: `log_ab_event()` (lines 1386–1405) writes a dict row to `ab_test_events.csv`, creating the header on first call.

2.2 Intervention (Version B)

The Cleaning tab in both arms exposes the same nine operations, same inputs, and same column selectors. Only the layout and microcopy differ.

Element	Version A (Control)	Version B (Treatment)
Sidebar intro	“This version keeps the original Cleaning interface.”	“This version highlights a four-step workflow and emphasizes the key action buttons for the A/B test.”
Version badge	“Version A / Control”	“Version B / Guided”

Element	Version A (Control)	Version B (Treatment)
Action buttons	Two flat buttons: `Preview` (`btn-outline-dark`) and `Apply` (`btn-dark`), side-by-side	A `clean-cta-box` containing the header **"Step 4 — Preview, then save"**, a note ("Use Preview Changes first. If the preview looks correct, finish with Apply and Save."), and two prominent buttons: `Preview Changes` (`btn-primary`, gradient blue) and `Apply and Save` (`btn-success`, green). 48px min-height.
Contextual hints	None	Action-specific hint lines (e.g., for `handle_missing`: "Select the columns with NA values, then choose whether to impute, fill, or drop them."). Changes as the user picks a different operation.

All other Cleaning inputs (strategy dropdown, k-NN parameters, outlier action, save-mode radio) and the rest of the app are identical by construction — we render the same underlying widgets with the same defaults in both arms.

2.3 Randomisation

Assignment is uniform random at session start: `random.choice(["A", "B"])`. The assignment is stored in a Shiny reactive value scoped to the session, so it remains stable across the user's interactions within one browser session. Assignment is **not** persisted across sessions — a user who returns tomorrow may flip groups. We address this as a limitation in §6.

2.4 Primary and secondary metrics

All metrics are computed at the session level (one observation per `session_id`).

#	Metric	Definition	Scale	Test
1 (primary)	`apply_rate`	`n_apply_events / max(n_preview_events + n_apply_events, 1)`	[0,1] continuous	Welch's t-test + Mann-Whitney U

#	Metric	Definition	Scale	Test
2 (secondary)	`preview_to_apply_conversion`	indicator: 1 if session had ≥1 preview AND ≥1 apply, 0 otherwise	binary	Two-proportion z-test
3 (secondary)	`apply_success_rate`	`n_apply_success / max(n_apply, 1)` among sessions with ≥1 apply	[0,1] continuous	Welch's t-test
4 (secondary)	`successful_actions_per_session`	count of `apply_clean` rows with `success=True` per session	integer ≥ 0	Mann-Whitney U

Multiple comparisons. Bonferroni correction over four tests: reject H0 at family-wise $\alpha = 0.05$ only if $p < 0.0125$.

Effect size. Cohen's *d* for continuous metrics; absolute and relative lift for the binary metric. Ninety-five-percent confidence intervals on the effect are estimated by 10 000 bootstrap resamples.

2.5 Event schema, blinding, and retrieval

Blinding. Users are not told which arm they are in. The `ab_group` is kept in server-side state and written only to the log. The Cleaning-tab sidebar shows the same neutral "Preview before applying" tip to both arms; the treatment manifests only as the guided four-step layout and the prominent CTA box, not as a label. This eliminates the demand-characteristics risk of telling users they are a "Control" or a "Treatment".

Logging. Every preview or apply click writes one row to `ab_test_events.csv` (plain CSV, append-only). The writer is wrapped in try/except so that an I/O failure degrades silently and never crashes the user's action. Schema:

Column	Type	Notes
`timestamp`	datetime (YYYY-MM-DD HH:MM:SS)	local server time on Posit Cloud
`session_id`	uuid4 string	unique per browser session
`ab_group`	`A` or `B`	assignment
`event_type`	`preview_clean` or `apply_clean`	fired by the Cleaning tab only
`clean_action`	e.g. `handle_missing`, `remove_duplicates`, `scale_columns`, …	nine possible values

Column	Type	Notes
`dataset_key`	descriptive version key of the dataset being cleaned	e.g. `builtin_sleep`
`columns_count`	integer	columns selected at click time
`success`	`True` / `False` / `""`	`""` if the operation has not yet resolved
`seconds_since_session_start`	float	wall-clock seconds since the session opened the Cleaning tab
`details`	free-form string	summary message or exception text

Retrieval. The event CSV lives on the Posit Cloud container's filesystem. The Guide tab exposes a collapsible "Team only — download A/B event log" accordion, gated by a shared password, that yields the current log via a standard Shiny download handler. Only the team members who know the password can retrieve the file, and the download is a no-op when the log does not yet exist. This lets the team pull periodic snapshots without relying on Posit Cloud's container-level file access.

Consent. The Guide tab carries a short notice ("This app is part of a Columbia STAT 5243 class research project. Anonymous session interaction events (no personal data, no uploaded file content) are logged for the analysis. By using the app you consent to this research-purposes logging.") so users shared the link from Reddit, LinkedIn, and WeChat are informed that usage is being logged.

---

3. Data Collection

3.1 Source of traffic

The single deployment URL was shared in three channels, cumulative over the collection window:

1. **STAT 5243 classmates** — posted to the course Slack and announced in class.
2. **Personal networks** — direct messages to friends and LinkedIn contacts.
3. **Public** — social-media posts (Reddit r/datascience, X/Twitter, WeChat) inviting anyone to try a "free data-cleaning web tool and leave no account".

3.2 Collection window

- **Start:** 2026-04-18 09:30 EST (when `app_trt.py` went live on Posit Cloud)
- **Cutoff:** 2026-04-19 12:00 EST (≈12 hours before the 23:59 submission deadline, to leave time for analysis and report finalisation)

3.3 Sample

Sample sizes at cutoff (from `ab_test_events.csv`):

Group	Sessions	Preview events	Apply events	Sessions with ≥1 cleaning event
A (control)	{N_A}	{P_A}	{Ap_A}	{S_A}
B (treatment)	{N_B}	{P_B}	{Ap_B}	{S_B}

These counts are populated by `ab_analysis.py` on the frozen log at cutoff and inserted into this table.

3.4 Inclusion / exclusion criteria

- **Included:** every session with `session_id` appearing at least once in `ab_test_events.csv`.
- **Excluded:** sessions where the only events were errors with no successful action (treated as "no exposure to the intervention"). Sensitivity-analysis: we re-run the primary analysis including those sessions in §4 to check robustness.

3.5 Data quality checks

- **Randomisation balance.** Two-sided binomial test against H0: $p = 0.5$. Observed p = {bal_p}.
- **Timestamp sanity.** No rows outside the collection window.
- **Schema stability.** One CSV header row, consistent column count across all data rows.
- **No PII.** Only `session_id` (uuid) is stored; no IP, no cookie, no uploaded file content.

---

4. Statistical Analysis and Results

4.1 Method summary

For each metric we:

1. Aggregate events to one row per `session_id` (see schema in §2.4).
2. Run the specified parametric and/or non-parametric test between groups.
3. Estimate the effect size with Cohen's *d* (continuous) or proportion difference (binary).
4. Bootstrap a 95 % CI on the effect (10 000 resamples, percentile method).
5. Apply Bonferroni correction across the four tests.

The full pipeline is implemented in ab_analysis.py and is reproducible with:

```
python ab_analysis.py ab_test_events.csv --out figures/ --seed 20260418
```

4.2 Descriptive statistics

Metric	Group A mean (± sd)	Group B mean (± sd)	Lift
`apply_rate`	{mu_A_1} ± {sd_A_1}	{mu_B_1} ± {sd_B_1}	{lift_1}
`preview_to_apply_conversion`	{mu_A_2}	{mu_B_2}	{lift_2}
`apply_success_rate`	{mu_A_3} ± {sd_A_3}	{mu_B_3} ± {sd_B_3}	{lift_3}
`successful_actions_per_session`	{mu_A_4} ± {sd_A_4}	{mu_B_4} ± {sd_B_4}	{lift_4}

4.3 Inferential results

Metric	Test	Statistic	p-value (raw)	p-value (Bonferroni)	Effect size	95 % CI on effect	Decision at α_family=0.05
apply_rate (primary)	Welch's t	{t_1}	{p_1}	{p_1_adj}	d = {d_1}	[{lo_1}, {hi_1}]	{dec_1}
apply_rate (primary, non-parametric)	Mann-Whitney U	{U_1}	{pu_1}	{pu_1_adj}	—	—	{decu_1}
preview_to_apply_conversion	2-proportion z	{z_2}	{p_2}	{p_2_adj}	Δp = {dp_2}	[{lo_2}, {hi_2}]	{dec_2}
apply_success_rate	Welch's t	{t_3}	{p_3}	{p_3_adj}	d = {d_3}	[{lo_3}, {hi_3}]	{dec_3}
successful_actions_per_session	Mann-Whitney U	{U_4}	{p_4}	{p_4_adj}	—	—	{dec_4}

4.4 Figures

- **Figure 1** (`figures/apply_rate_by_group.png`) — histogram of per-session `apply_rate`, coloured by group, with group means and bootstrap 95 % CIs overlaid.
- **Figure 2** (`figures/preview_apply_funnel.png`) — per-group funnel chart: sessions, then sessions with at least one preview, sessions with at least one apply, and sessions with at least one successful apply.
- **Figure 3** (`figures/successful_actions_distribution.png`) — boxplot of successful-actions-per-session by group, overlaid with a swarm.

4.5 Power and sensitivity

- **Ex-post power** for the primary test at observed effect size and sample size: {power_1}.
- **Sensitivity to exclusion rule** — primary test p-values re-run including error-only sessions: {p_1_incl}.

---

5. Interpretation and Conclusion

{interp_paragraph_1}

Practical takeaways:

- {takeaway_1}
- {takeaway_2}
- {takeaway_3}

Roll-out recommendation. {rollout_recommendation}.

---

6. Challenges and Limitations

Randomisation is session-scoped, not user-scoped. Because we do not set a cookie and we do not use URL parameters, a returning visitor may end up in a different group on a second visit. This adds noise to the per-user treatment effect estimate but does not bias the session-level primary analysis since each session is drawn i.i.d. from Bernoulli(0.5) at start. A user-scoped design would have required a cookie or persistent login, which was out of scope for this course project.

Small and self-selected sample. The traffic we collected comes from classmates and personal networks. The population is younger, more technical, and more motivated than a typical end-user — external validity to real users of a generic data-cleaning product is therefore limited.

Single-session exposure. Users interact with the app once and then leave. We therefore capture *first-exposure* effects only; long-run learning and retention effects are not measurable with this design.

Cleaning tab is one of six. Not every session reaches the Cleaning tab. Sessions that never fire a `preview_clean` or `apply_clean` event are absent from the log entirely, so we cannot estimate a "cleaning-tab visit rate" from this data.

Timestamp granularity. Events are logged with one-second granularity. Rapid clicks inside a single second can appear out of order on replay, though this does not affect any of our aggregate metrics.

No pre-registration. Hypotheses and metrics were finalised inside the team before deployment but were not registered externally. The family of four tests was fixed before data cutoff; Bonferroni correction is reported.

Single experimental run. We ran the test once, over ~26 hours, with no replication or A/A control arm. An A/A run would have been a good sanity check but was not feasible in the project timeline.

Appendix A. Team Contributions

Team Member	Contribution
Zeming Liang	Treatment app (`app_trt.py`): in-app A/B randomisation, event logger, guided Cleaning-tab UI (CTA box, contextual hints). Posit Cloud deployment. Feature-engineering and EDA backend consistency across arms. Synthetic seed-data generator for pipeline testing. EDA slide deck.
Bohong Zheng	Statistical analysis pipeline (`ab_analysis.py`), figures, ex-post power calculation.
Maya Rubin	Cleaning backend consistency across arms. A/B unit tests in `tests.py` (log schema, randomisation balance, pipeline end-to-end).
Zuer Weng	Report assembly.

Appendix B. Reproducibility

```
# Clone and install
git clone https://github.com/ZemingLiang/STAT5243-Project3-Team21.git
cd STAT5243-Project3-Team21
pip install -r requirements.txt

# Run the A/B app locally
shiny run app_trt.py              # opens http://127.0.0.1:8000 with random A/B assignm

# Run the analysis on a collected log
python ab_analysis.py ab_test_events.csv --out figures/ --seed 20260418

# Run tests
python tests.py                    # unit tests across modules + A/B-specific tests
```

Dataset used for the cleaning workflow: `test_data/sleep_mobile_stress_dataset_15000.csv` (15 000 rows × 13 columns). Log output: `ab_test_events.csv` (appended per session).